VETRA

Delivery System

Updated October 2026

How a parcel gets from a vendor to a buyer: who touches it, what each person sees, what the server enforces, and how money moves at the end. Everything here was read out of the code it describes.

Piece	Where
State machine, fees, assignment, escrow, sweep	`backend/src/utils/delivery.js`
HTTP API	`backend/src/routes/delivery.routes.js`, mounted at `/api/delivery`
Checkout and vendor shipment changes	`backend/src/routes/orders.routes.js`
Schema	`backend/migrations/015_delivery_system.sql`
Scheduled sweep	`backend/scripts/delivery-cron.js`
Buyer: delivery form, quotes	`customer/cart.html`, `customer/assets/cart.js`
Buyer: live tracking panel, confirm receipt	`customer/assets/orders.js`
Vendor: ready / ship myself / pickup handover / returns	`vendor/assets/orders.js`
Rider app	`rider/login.html`, `rider/index.html`, `rider/assets/rider.js`
Dispatch console	`admin/deliveries.html`, `admin/assets/deliveries.js`
Public tracking	`track.html`, `track.js`

1. The people involved

- **Buyer.** Chooses delivery or pickup at checkout, holds the order's 6-digit **delivery code**, and confirms receipt.
- **Vendor.** Packs the order, marks it ready, hands it to the rider by reading out a 4-digit **pickup code**, and takes back returns.
- **Rider.** A new account type (`users.role = 'rider'`). Riders never sign up themselves; an admin creates them, and they set a password from an emailed link. Each rider works one state (`users.state`), which they choose themselves in the rider app (refused while they hold any job, offers included), has a vehicle and plate, and toggles on and off duty.

- **Dispatch.** Admins in the Deliveries console. Support admins can watch; Moderators and Super Admins can assign, cancel, and manage riders; only Super Admins can change rates and settings or record rider payouts.

2. Two codes, two handovers

Every handover is proven by a code that only the *receiving* side's counterparty holds, so nobody can mark a step done on their own.

Handover	Code	Who sees it	Who types it
Vendor → rider	4-digit `deliveries.pickup_code`	Vendor (Orders page), admin	Rider
Rider → buyer	6-digit `orders.delivery_code`	Buyer only (Orders page, order emails)	Rider
Vendor → buyer (pickup orders)	same 6-digit `orders.delivery_code`	Buyer only	Vendor

The buyer's code is stripped from every vendor, rider and admin response (`GET /orders/vendor`, the admin order lists, the rider job list). A vendor who could read it could mark an order delivered without handing anything over. Wrong guesses are counted in `orders.delivery_code_attempts`; after 5 the order locks and dispatch has to unlock it after checking with the buyer by phone. Code entry endpoints are also rate-limited per IP.

3. Checkout and fees

The cart page asks for delivery or pickup. For delivery it takes address, state, phone and optional directions, prefilled from the buyer's profile. A cart with items from several stores becomes one order per store, and each store's order ships separately, so each gets its own quote from `GET /api/delivery/quote?vendorId&state&subtotal`.

The fee comes from `delivery_rates`, one row per destination state:

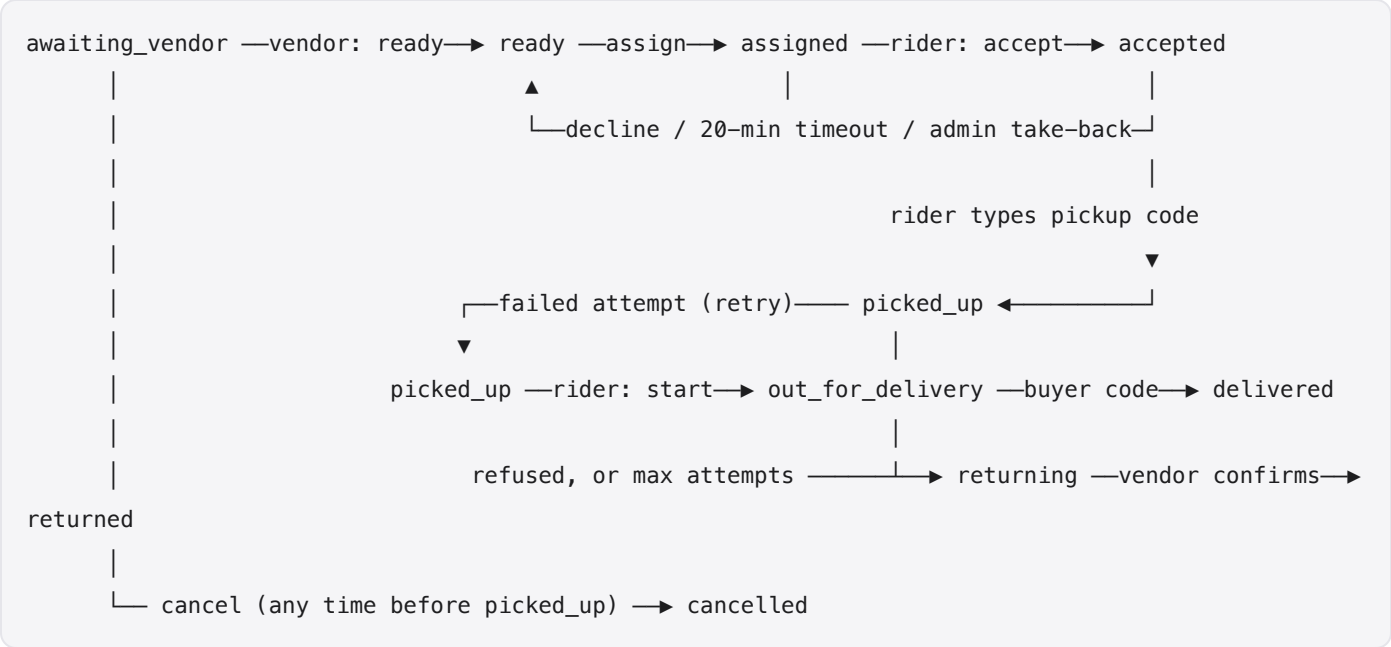
- **same-state fee** when the vendor's `users.state` equals the destination,
- **inter-state fee** otherwise,
- an ETA window (same-state is always quoted 1–2 days),
- an `active` flag. Turning a state off makes checkout reject delivery there and point the buyer to pickup.

Defaults: ₦1,500 same-state (₦2,000 in Lagos, FCT and Rivers) and ₦3,000–₦5,000 between states. Super Admins edit them in Deliveries → Rates & settings.

`platform_settings.free_delivery_threshold` waives the fee once a store's subtotal reaches it (0 turns this off). The rider is still paid in full; VETRA absorbs it.

Checkout (`POST /api/payments/checkout` , see `PAYMENTS.md`) recomputes the quote itself and never trusts the cart's number. The order stores `delivery_fee` separately from `total` , so `total` keeps meaning the vendor's goods everywhere it was already summed (vendor earnings, admin stats). The buyer pays `total + delivery_fee` , plus a processing fee covering Paystack. Orders are created only once the payment is confirmed, and delivery orders get a `deliveries` row in the same transaction as the order, with **snapshots** of the pickup address (the vendor's profile) and drop-off address, so a later profile edit can't reroute a parcel already moving. `rider_payout` is fixed at this point: `platform_settings.rider_payout_percent` (80% by default) of the zone rate.

4. The delivery lifecycle



Each step mirrors onto `orders.status` , which every older page already reads:

`deliveries.status`	`orders.status`
ready, assigned, accepted	processing
picked_up, returning	shipped
out_for_delivery	out_for_delivery
delivered	completed (escrow still held; the rider is paid now)
returned, cancelled	cancelled, escrow refunded, buyer refunded to their card

Every transition is one guarded `UPDATE ... WHERE id = ? AND status IN (...)` , with `affectedRows` checked. Two riders accepting at once, or a vendor cancelling while a rider picks up, can't both succeed: the loser gets a 409. Every transition also appends to `delivery_events` , which drives every timeline. Each event

has a detailed `note` for vendors and dispatch and a friendlier `public_note` for buyers and the public tracking page.

Step by step

1. **Vendor packs.** The order row shows **Packed, ready for pickup**, **I'll ship it myself**, and **Cancel order**.
2. **Ready.** With `auto_assign_riders` on (the default), the server immediately offers the job to the on-duty, active rider in the vendor's state with the fewest live jobs (max 5 each), skipping anyone who already declined it or let it time out. If nobody qualifies, the parcel waits in the dispatch queue ("Needs rider") and the sweep retries. A rider going on duty also triggers an immediate match for waiting parcels in their state. With auto-assign off, dispatch assigns every job by hand.
3. **Offer.** The rider sees pickup details, destination state and payout, but **not** the buyer's name, address or phone until they accept. They have 20 minutes; after that the sweep returns the job to the queue.
4. **Accepted.** The buyer and vendor are told who's coming. The vendor's row shows the rider's name, phone and plate, and the pickup code to read out.
5. **Picked up.** The rider types the vendor's code. The order becomes `shipped`.
6. **Out for delivery.** The rider taps **Start delivery**. The buyer gets a notification and an email with their delivery code and the rider's phone. The rider app sends the phone's GPS position every 30 seconds while the rider has a live job, so the buyer's panel and `track.html` show a map, refreshed every 30 seconds. A position older than 15 minutes is hidden rather than shown as current.
7. **Delivered.** The rider types the buyer's 6-digit code, optionally attaching a handover photo (uploaded to Cloudinary). The order becomes `completed`, `delivered_at` is stamped, and escrow stays **held**. The rider's payout is created at this moment and sent by Paystack transfer (see `PAYMENTS.md` §3).
8. **Failed attempt.** The rider picks a reason (unreachable, nobody home, wrong address, refused, other) with an optional note. The buyer is told and the rider keeps the parcel to retry. A refusal, or reaching `max_delivery_attempts` (3 by default), switches the delivery to `returning`.
9. **Return.** The rider takes it back; the vendor taps **I've got it back**. The order is cancelled, escrow marked refunded, the buyer refunded to their card, and the items go back into stock. The rider is still paid for the job, automatically.

Variations

- **Pickup orders** have no `deliveries` row and no fee. The vendor accepts the order, then types the buyer's 6-digit code at the counter (**Confirm pickup**) to complete it. Vendors can't mark a pickup order completed any other way.
- **"I'll ship it myself"** switches the delivery to `mode = 'self'` (allowed until a rider has the parcel), notifies any rider who'd been offered it, and gives the vendor back the old **Update shipment** modal (carrier, tracking number, status), mirrored onto the delivery row. The buyer still has to confirm receipt, or wait out the window, before escrow releases.
- **Orders placed before this system** have no `delivery_code` and keep the old manual flow exactly.

5. Escrow release

`buyer-protection.html` promised that funds release when the buyer confirms delivery, or automatically 48 hours after drop-off, and that a reported problem freezes them. That is now what the code does:

- **Buyer confirms.** On a delivered order, the buyer's panel shows **I received it, all good** and the auto-release time. Tapping it calls `POST /api/delivery/orders/:id/confirm`, which releases escrow and notifies the vendor.
- **Window passes.** The sweep releases any `completed` order whose `delivered_at` is older than `auto_confirm_hours` (48 by default).
- **Dispute.** If the order has an **open** report (the existing "Report an issue" button), the sweep skips it. Once an admin resolves or dismisses the report, the next sweep releases it.

Vendor marking an order delivered (self-ship) no longer releases escrow on the spot either. Cancelling an order marks its escrow `refunded` and refunds the buyer.

Escrow is real money now: the buyer paid through Paystack at checkout, and releasing escrow sends the vendor their payout (goods minus commission) by Paystack transfer. See `PAYMENTS.md`.

6. The sweep

`scripts/delivery-cron.js` (`npm run cron`) runs `runDeliveryCron()`:

1. release escrow on orders past the protection window with no open report,
2. return rider offers older than 20 minutes to the queue,
3. retry auto-assign for every parcel still waiting.

On cPanel it runs from Cron Jobs every 10 minutes (exact line in `backend/README.md`). On a host without cron, set `CRON_SECRET` and have an external scheduler `POST /api/delivery/cron` with header `x-cron-secret`. Every step is a guarded update, so overlapping or repeated runs are safe.

7. Dispatch console (admin/deliveries.html)

- **Stats:** parcels needing a rider, in progress, returning, locked codes, riders on duty, auto-assign on or off.
- **Dispatch:** tabs for Needs rider, Offered, In progress, Returning, Vendor packing, Finished and All, plus pickup-state filter and search by order ID (`VTA-…`; an old `VTR-` reference from a pre-switch email also matches). Row actions depend on status:
- **Assign / Reassign.** Riders are listed in the vendor's state first, on-duty first, least busy first.
- **Take back.** Returns the job to the queue.
- **Cancel.** Only before pickup; requires a reason.
- **Send back.** Returns an in-flight parcel to the vendor; requires a reason.
- **Unlock code.**

- **Timeline.** Every event with who did it, the pickup code, the rider's last position, the proof photo, and when escrow auto-releases.
- The view refreshes every 30 seconds.
- **Riders:** add a rider (name, email, phone, state, vehicle, plate), which emails them a 72-hour set-password link. Also edit, suspend (signs them out immediately, and any job they haven't collected returns to the queue), reactivate, resend the invite, and **Mark paid**. Riders are paid automatically by Paystack transfer for every job on a paid order; Mark paid only covers jobs from before online payments, recording a bank transfer made outside the app.
- **Rates & settings:** the per-state fee table and the five delivery settings.

8. Rider app (/rider/)

Built for a phone on the road: one column, large buttons, the numeric keypad for codes.

- **Duty switch and stats:** today's earnings, earnings waiting to be paid out, deliveries completed. Below it, the **Delivery area** picker. A rider can't go off duty with live jobs.
- **Jobs tab:** each job card shows exactly the next action for its status, with **Call** and Google Maps **Directions** for pickup and drop-off. It re-polls every 20 seconds for new offers without clobbering a code the rider is typing.
- **History tab:** completed and returned jobs, each with its payout status: Paid, Sending, Pending (automatic payouts off), Add your bank, or Retrying.
- **Bank tab:** the account payouts go to. The name comes from the bank, never typed; a red dot shows until one is saved.
- Location sharing is on only while on duty with a live job; if the browser denies location, the app says so.

9. Notifications and emails

Every step notifies the parties it concerns in-app (`notifications.type = 'delivery'`). Emails go out where someone would want one:

- **Buyer:** order confirmation with the code, out for delivery (code and rider's phone), delivered (with the protection window), missed attempt, returning, returned, cancelled. Each email links to `track.html`.
- **Rider:** each new job offer.
- **Vendor:** in-app only for delivery steps, plus the existing new-order email.

10. API

All under `/api/delivery`.

Route	Who	Does
`GET /quote?vendorId&state&subtotal`	public	Fee, ETA, deliverable, rider payout
`GET /track/:code`	public	Status, public timeline; rider first name and live position while en route. No address, phone, buyer name or code
`GET /orders/:orderId`	buyer, vendor, admin	Delivery detail; buyer sees their code, vendor and admin see the pickup code
`POST /orders/:orderId/confirm`	buyer	Confirm receipt, release escrow
`POST /orders/:orderId/ready`	vendor	Packed; triggers auto-assign
`POST /orders/:orderId/self-ship`	vendor	Switch to own courier (before pickup)
`POST /orders/:orderId/pickup-complete`	vendor	Pickup order handover, `{ code }`
`POST /orders/:orderId/return-received`	vendor	Returned parcel received; cancels, restocks
`GET /rider/me`, `PATCH /rider/duty`	rider	Profile and earnings; `{ onDuty }`
`PATCH /rider/zone`	rider	`{ state }`, the rider's own delivery area; 409 while they hold any job or offer
`GET /rider/jobs`, `GET /rider/history`	rider	Live jobs; finished jobs
`POST /rider/jobs/:id/accept` · `decline` · `pickup` · `depart` · `deliver` · `fail`	rider	`pickup {pickupCode}`, `deliver {code, photoUrl?}`, `fail {reason, note?}`, `decline {reason?}`
`POST /rider/location`	rider	`{ lat, lng }`, stored only on jobs the rider is moving
`GET /admin/overview`, `GET /admin/deliveries?group&state&q`	admin	Stats; dispatch list
`POST /admin/deliveries/:id/assign` · `unassign` · `cancel` · `return` · `unlock-code`	Moderator+	Dispatch actions
`GET/POST /admin/riders`, `PATCH /admin/riders/:id`, `POST /admin/riders/:id/resend-invite`	view: admin; change: Moderator+	Rider management
`POST /admin/riders/:id/payouts`	Super Admin	Mark paid, for jobs with no automatic payout
`GET/PUT /rider/payout-account`, `/banks`, `/resolve`	rider	Payout bank account

Route	Who	Does
`GET /admin/rates`, `PUT /admin/rates/:state`, `PATCH /admin/settings`	view: admin; change: Super Admin	Rates and settings
`POST /cron`	`x-cron-secret`	The sweep, when `CRON_SECRET` is set

Riders sign in through the existing `POST /api/auth/signin` with `role: "rider"`. `PATCH /api/auth/me` won't change a rider's state; their dispatch zone moves only through `PATCH /rider/zone`.

11. Data model

- `users`: role gains `rider`; new `rider_vehicle`, `rider_plate`, `rider_on_duty`.
- `delivery_rates`: one row per state, seeded for all 37.
- `platform_settings`: `free_delivery_threshold`, `rider_payout_percent`, `auto_assign_riders`, `auto_confirm_hours`, `max_delivery_attempts`.
- `orders`: `delivery_fee`, `delivery_state`, `delivery_phone`, `delivery_notes`, `delivery_code`, `delivery_code_attempts`, `buyer_confirmed_at`.
- `deliveries`: one per delivery-method order. Mode, status, rider, pickup code, address snapshots, fee charged, rider payout and paid-at, failed attempts and last reason, proof photo, last rider position, and a timestamp per step.
- `delivery_events`: append-only timeline with `note`, `public_note`, actor and role.

12. Known limits

- **Polling, not push.** The rider app, the buyer's panel and dispatch refresh every 20–30 seconds; nothing is real-time.
- **Location is last-known only.** There's no route history, and no ETA calculated from position.
- **One parcel per job.** There's no batching of several orders onto one route, and no line-haul between states: an inter-state order is picked up by a rider in the vendor's state, who delivers it.
- **Payouts depend on the Paystack balance.** Riders and vendors are paid from it; if it's short, transfers fail and retry (`PAYMENTS.md` §3).
- **The 20-minute offer timeout is a constant** in `utils/delivery.js` (`ASSIGNMENT_TIMEOUT_MINUTES`), as are the 5-job cap and the 5-attempt code lock.